솔루션 설계를 검토합니다

BeeGFS on NetApp with E-Series Storage

NetApp
October 13, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/beegfs/beegfs-design-overview.html on October 13, 2022. Always check docs.netapp.com for the latest.

목차

솔루션 설계를 검토합니다 1
설계 개요 1
하드웨어 구성 1
소프트웨어 구성 2
설계 검증 6
사이징 지침 8
성능 튜닝 10

솔루션 설계를 검토합니다

설계 개요

BeeGFS 병렬 파일 시스템을 NetApp EF600 스토리지 시스템과 결합하는 NetApp 솔루션 기반 BeeGFS를 지원하려면 특정 장비, 케이블링, 구성이 필요합니다.

자세한 내용:

- "하드웨어 구성"
- "소프트웨어 구성"
- "설계 검증"
- "사이징 지침"
- "성능 튜닝"

하드웨어 구성

NetApp의 BeeGFS에 대한 하드웨어 구성에는 파일 노드 및 네트워크 케이블 연결이 포함됩니다.

파일 노드 구성

파일 노드에는 동일한 수의 PCIe 슬롯 및 메모리에 대한 로컬 액세스를 포함하는 별도의 NUMA 존으로 구성된 2개의 CPU 소켓이 있습니다.

InfiniBand 어댑터는 적절한 PCI 라이저 또는 슬롯에 설치되어야 사용 가능한 PCIe 레인 및 메모리 채널에 걸쳐 작업 부하가 분산됩니다. 개별 BeeGFS 서비스에 대한 작업을 특정 NUMA 노드에 완전히 격리하여 워크로드의 균형을 조정합니다. 목표는 두 개의 독립적인 단일 소켓 서버처럼 각 파일 노드에서 유사한 성능을 얻는 것입니다.

다음 그림에서는 파일 노드 NUMA 구성을 보여 줍니다.

[]

BeeGFS 프로세스는 사용된 인터페이스가 동일한 존에 있도록 특정 NUMA 존에 고정됩니다. 이렇게 구성하면 소켓 간 연결을 통한 원격 액세스가 필요하지 않습니다. 소켓 간 연결은 QPI 또는 GMI2 링크라고도 합니다. 최신 프로세서 아키텍처에서도 HDR InfiniBand와 같은 고속 네트워킹을 사용할 때 병목 현상이 발생할 수 있습니다.

네트워크 케이블 연결 구성

구성 요소 내에서 각 파일 노드는 총 4개의 중복 InfiniBand 연결을 사용하여 2개의 블록 노드에 연결됩니다. 또한 각 파일 노드에는 InfiniBand 스토리지 네트워크에 대한 4개의 이중화된 접속이 있습니다.

다음 그림에서 주목하십시오.

- 녹색으로 표시된 모든 파일 노드 포트는 스토리지 패브릭에 접속하는 데 사용되며, 다른 모든 파일 노드 포트는 블록 노드에 직접 연결됩니다.

- 특정 NUMA 존에 있는 2개의 InfiniBand 포트는 동일한 블록 노드의 A 및 B 컨트롤러에 연결됩니다.
- NUMA 노드 0의 포트는 항상 첫 번째 블록 노드에 연결됩니다.
- NUMA 노드 1의 포트는 두 번째 블록 노드에 연결됩니다.

[]

> 이중화 스위치가 있는 스토리지 네트워크의 경우 녹색으로 표시된 포트가 한 스위치에 연결되고 어두운 녹색으로 표시된 포트는 다른 스위치에 연결해야 합니다.

그림에 표시된 케이블 연결 구성을 통해 각 BeeGFS 서비스는 다음을 수행할 수 있습니다.

- BeeGFS 서비스를 실행 중인 파일 노드에 관계없이 동일한 NUMA 존에서 실행합니다.
- 장애 발생 위치에 관계없이 프런트엔드 스토리지 네트워크와 백엔드 블록 노드에 대한 보조 최적 경로 제공
- 블록 노드의 파일 노드 또는 컨트롤러에 유지 관리가 필요한 경우 성능 영향을 최소화합니다.

PCIe 양방향 대역폭을 최대한 활용하려면 각 InfiniBand 어댑터의 포트 하나를 스토리지 패브릭에 연결하고 다른 포트는 블록 노드에 연결해야 합니다. HDR InfiniBand 포트의 이론적인 최대 속도는 25GBps입니다(신호 및 기타 오버헤드는 고려하지 않음). PCIe 4.0 x16 슬롯의 최대 단일 방향 대역폭은 32GBps이며 이론적으로 50GBps 대역폭을 처리할 수 있는 이중 포트 InfiniBand 어댑터가 통합된 파일 노드를 구현할 때 잠재적인 병목 현상이 발생합니다.

다음 그림은 전체 PCIe 양방향 대역폭을 활용하는 데 사용되는 케이블링 설계를 보여줍니다.

[]

각 BeeGFS 서비스에 대해 동일한 어댑터를 사용하여 클라이언트 트래픽에 사용되는 기본 포트를 해당 서비스 볼륨의 기본 소유자인 블록 노드 컨트롤러의 경로와 연결합니다. 자세한 내용은 을 참조하십시오 "소프트웨어 구성".

소프트웨어 구성

NetApp 기반 BeeGFS에 대한 소프트웨어 구성에는 BeeGFS 네트워크 구성 요소, EF600 블록 노드, BeeGFS 파일 노드, 리소스 그룹, BeeGFS 서비스가 포함됩니다.

BeeGFS 네트워크 구성

BeeGFS 네트워크 구성은 다음과 같은 구성 요소로 이루어집니다.

- * 부동 IP * 부동 IP는 동일한 네트워크의 모든 서버로 동적으로 라우팅될 수 있는 일종의 가상 IP 주소입니다. 여러 서버가 동일한 부동 IP 주소를 소유할 수 있지만, 특정 시간에 한 서버에서만 활성화될 수 있습니다.

  각 BeeGFS 서버 서비스에는 BeeGFS 서버 서비스의 실행 위치에 따라 파일 노드 간에 이동할 수 있는 고유한 IP 주소가 있습니다. 이러한 부동 IP 구성을 통해 각 서비스가 다른 파일 노드로 독립적으로 페일오버할 수 있습니다. 클라이언트는 특정 BeeGFS 서비스의 IP 주소를 알고 있으면 됩니다. 이 경우 현재 해당 서비스를 실행 중인 파일 노드를 알 필요가 없습니다.

- * BeeGFS 서버 다중 홈 구성 * 솔루션의 밀도를 높이기 위해 각 파일 노드에는 동일한 IP 서브넷에 구성된 IP를 가진 여러 스토리지 인터페이스가 있습니다.

  기본적으로 하나의 인터페이스에 대한 요청은 동일한 서브넷에 있는 경우 다른 인터페이스에서 응답할 수 있기

때문에 Linux 네트워킹 스택에서 이 구성이 예상대로 작동하는지 확인하기 위해 추가 구성이 필요합니다. 다른 단점 외에도 이 기본 동작으로 인해 RDMA 연결을 적절하게 설정하거나 유지할 수 없습니다.

Ansible 기반 배포에서는 부동 IP가 시작 및 중지되는 시기와 함께 RP(역방향 경로) 및 ARP(주소 해상도 프로토콜) 동작 조임을 처리하고, 다중 홈 네트워크 구성이 제대로 작동하도록 해당 IP 경로 및 규칙을 동적으로 생성합니다.

- * BeeGFS 클라이언트 다중 레일 구성 * _Multi-RAIL_은 애플리케이션이 여러 개의 독립적인 네트워크 "레일"을 사용하여 성능을 향상시키는 기능을 의미합니다.

  BeeGFS는 RDMA 연결을 위해 RDMA를 사용할 수 있지만, BeeGFS는 IPoIB를 사용하여 RDMA 연결 검색 및 설정을 간소화합니다. BeeGFS 클라이언트가 여러 InfiniBand 인터페이스를 사용할 수 있도록 하려면 각 클라이언트를 다른 서브넷에 있는 IP 주소로 구성한 다음 각 서브넷에 있는 BeeGFS 서버 서비스의 절반에 대해 기본 인터페이스를 구성할 수 있습니다.

  다음 다이어그램에서는 밝은 녹색으로 강조 표시된 인터페이스가 하나의 IP 서브넷(예: 100.127.0.0/16)에 있고 어두운 녹색 인터페이스는 다른 서브넷(예: 100.128.0.0/16)에 있습니다.

  다음 그림에서는 여러 BeeGFS 클라이언트 인터페이스에서 트래픽의 균형을 조정하는 방법을 보여 줍니다.

  []

  BeeGFS의 각 파일은 일반적으로 여러 스토리지 서비스에 걸쳐 스트라이핑되기 때문에 다중 레일 구성을 통해 클라이언트는 단일 InfiniBand 포트보다 더 많은 처리량을 달성할 수 있습니다. 예를 들어, 다음 코드 샘플은 클라이언트가 두 인터페이스 간에 트래픽의 균형을 조정할 수 있도록 하는 일반적인 파일 스트라이핑 구성을 보여 줍니다.

```
root@ictad21h01:/mnt/beegfs# beegfs-ctl --getentryinfo myfile
Entry type: file
EntryID: 11D-624759A9-65
Metadata node: meta_01_tgt_0101 [ID: 101]
Stripe pattern details:
+ Type: RAID0
+ Chunksize: 1M
+ Number of storage targets: desired: 4; actual: 4
+ Storage targets:
  + 101 @ stor_01_tgt_0101 [ID: 101]
  + 102 @ stor_01_tgt_0101 [ID: 101]
  + 201 @ stor_02_tgt_0201 [ID: 201]
  + 202 @ stor_02_tgt_0201 [ID: 201]
```

  두 개의 IPoIB 서브넷을 사용하는 것은 논리적인 구분입니다. 필요한 경우 단일 물리적 InfiniBand 서브넷(스토리지 네트워크)을 사용할 수 있습니다.

> ⓘ 단일 IPoIB 서브넷에서 여러 IB 인터페이스를 사용할 수 있도록 BeeGFS 7.3.0에 멀티 레일 지원이 추가되었습니다. BeeGFS 7.3.0을 GA 전에 NetApp 기반 BeeGFS 솔루션의 설계가 개발되었으며, BeeGFS 클라이언트에서 두 개의 IB 인터페이스를 사용하는 두 개의 IP 서브넷을 보여 줍니다. 다중 IP 서브넷 접근 방식의 한 가지 장점은 BeeGFS 클라이언트 노드에서 멀티호밍을 구성할 필요가 없기 때문입니다(자세한 내용은 참조) "BeeGFS RDMA 지원")를 클릭합니다.

EF600 블록 노드 구성

블록 노드는 동일한 드라이브 세트에 대한 공유 액세스를 가진 2개의 액티브/액티브 RAID 컨트롤러로 구성됩니다. 일반적으로 각 컨트롤러는 시스템에 구성된 볼륨의 절반을 소유하지만 필요에 따라 다른 컨트롤러를 인수할 수 있습니다.

파일 노드의 다중 경로 소프트웨어는 각 볼륨에 대한 최적화된 활성 경로를 결정하고 케이블, 어댑터 또는 컨트롤러에 장애가 발생할 경우 대체 경로로 자동으로 이동합니다.

다음 다이어그램은 EF600 블록 노드의 컨트롤러 레이아웃을 보여 줍니다.

[]

공유 디스크 HA 솔루션을 지원하기 위해 볼륨은 두 파일 노드에 매핑되므로 필요에 따라 서로 테이크오버할 수 있습니다. 다음 다이어그램은 BeeGFS 서비스 및 기본 볼륨 소유권이 최대 성능을 위해 구성되는 방법의 예를 보여 줍니다. 각 BeeGFS 서비스 왼쪽에 있는 인터페이스는 클라이언트 및 기타 서비스가 연락하는 데 사용하는 기본 인터페이스를 나타냅니다.

[]

앞의 예에서 클라이언트 및 서버 서비스는 인터페이스 i1b를 사용하여 스토리지 서비스 1과 통신하는 것을 선호합니다. 스토리지 서비스 1은 인터페이스 i1a를 기본 경로로 사용하여 첫 번째 블록 노드의 컨트롤러 A에 있는 해당 볼륨(storage_tgt_101, 102)과 통신합니다. 이 구성은 InfiniBand 어댑터에서 사용할 수 있는 양방향 PCIe 대역폭을 완벽하게 사용하고 PCIe 4.0에서 사용할 수 있는 것보다 듀얼 포트 HDR InfiniBand 어댑터에서 더 나은 성능을 제공합니다.

BeeGFS 파일 노드 구성

BeeGFS 파일 노드는 HA(High-Availability) 클러스터로 구성되어 여러 파일 노드 간에 BeeGFS 서비스의 페일오버를 지원합니다.

HA 클러스터 설계는 널리 사용되는 두 가지 Linux HA 프로젝트, 즉 클러스터 멤버십을 위한 Corosync 및 클러스터 리소스 관리를 위한 Pacemaker를 기반으로 합니다. 자세한 내용은 을 참조하십시오 "고가용성 애드온을 위한 Red Hat 교육".

NetApp은 클러스터가 지능적으로 BeeGFS 리소스를 시작하고 모니터링할 수 있도록 여러 OCF(Open Cluster Framework) 리소스 에이전트를 저술하고 확장했습니다.

BeeGFS HA 클러스터

일반적으로, HA를 사용하거나 사용하지 않고 BeeGFS 서비스를 시작할 때 다음과 같은 몇 가지 리소스를 사용해야 합니다.

- 서비스에 연결할 수 있는 IP 주소이며 일반적으로 Network Manager에서 구성합니다.
- BeeGFS에서 데이터를 저장하기 위한 타겟으로 사용되는 기본 파일 시스템입니다.

  일반적으로 이러한 항목은 '/etc/fstab'에 정의되어 있으며 systemd에 의해 마운트됩니다.

- 다른 리소스가 준비되면 BeeGFS 프로세스를 시작하는 시스템 서비스입니다.

  추가 소프트웨어가 없으면 이러한 리소스는 단일 파일 노드에서만 시작됩니다. 따라서 파일 노드가 오프라인이 되면 BeeGFS 파일 시스템의 일부를 액세스할 수 없습니다.

여러 노드에서 각 BeeGFS 서비스를 시작할 수 있으므로, Pacemaker는 각 서비스와 종속 리소스가 한 번에 하나의 노드에서만 실행되도록 해야 합니다. 예를 들어, 두 노드가 동일한 BeeGFS 서비스를 시작하려고 하면 둘 다 기본 타겟의 동일한 파일에 쓰려고 하면 데이터 손상이 발생할 위험이 있습니다. 이러한 시나리오를 피하기 위해, 페이스 메이커의 Corosync를 사용하여 전체 클러스터의 상태를 모든 노드에 걸쳐 안정적으로 유지하고 쿼럼을 설정합니다.

클러스터에서 장애가 발생하면 심장박동기가 반응하여 다른 노드에서 BeeGFS 리소스를 다시 시작합니다. 일부 시나리오에서는 심박조율기가 장애가 발생한 원래 노드와 통신하지 못하여 리소스가 중지되었는지 확인할 수 없습니다. 다른 곳에서 BeeGFS 리소스를 다시 시작하기 전에 노드가 다운되었는지 확인하려면 심장박동기가 장애가 있는 노드를 분리합니다. 즉, 전원을 제거하는 것이 좋습니다.

심박조율기가 PDU(Power Distribution Unit)를 사용하여 노드를 펜싱하거나 서버 BMC(Baseboard Management Controller)를 Redfish와 같은 API와 함께 사용하여 오픈 소스 펜싱 에이전트를 많이 사용할 수 있습니다.

BeeGFS가 HA 클러스터에서 실행 중인 경우 모든 BeeGFS 서비스 및 기본 리소스는 리소스 그룹의 페이스 메이커를 통해 관리됩니다. 각 BeeGFS 서비스 및 해당 서비스가 의존하는 리소스가 리소스 그룹으로 구성되어 리소스가 올바른 순서로 시작 및 중지되어 동일한 노드에 배치됩니다.

각 BeeGFS 리소스 그룹에 대해 심장박동기는 특정 노드에서 BeeGFS 서비스에 더 이상 액세스할 수 없을 때 장애 조건을 감지하고 페일오버를 지능적으로 트리거하는 사용자 지정 BeeGFS 모니터링 리소스를 실행합니다.

다음 그림에서는 심장박동기 제어 BeeGFS 서비스 및 종속성을 보여 줍니다.

[]

> ⓘ 동일한 유형의 여러 BeeGFS 서비스를 동일한 노드에서 시작할 수 있도록 다중 모드 구성 방법을 사용하여 BeeGFS 서비스를 시작하도록 페이스 메이커를 구성합니다. 자세한 내용은 를 참조하십시오 "멀티 모드에 대한 BeeGFS 문서".

BeeGFS 서비스는 여러 노드에서 시작할 수 있어야 하므로 각 서비스의 구성 파일('/etc/beegfs'에 있음)은 해당 서비스의 BeeGFS 타겟으로 사용되는 E-Series 볼륨 중 하나에 저장됩니다. 따라서 특정 BeeGFS 서비스에 대한 데이터와 함께 서비스를 실행해야 하는 모든 노드에서 해당 구성을 액세스할 수 있습니다.

```
# tree stor_01_tgt_0101/ -L 2
stor_01_tgt_0101/
├── data
│   ├── benchmark
│   ├── buddymir
│   ├── chunks
│   ├── format.conf
│   ├── lock.pid
│   ├── nodeID
│   ├── nodeNumID
│   ├── originalNodeID
│   ├── targetID
│   └── targetNumID
└── storage_config
    ├── beegfs-storage.conf
    ├── connInterfacesFile.conf
    └── connNetFilterFile.conf
```

설계 검증

BeeGFS on NetApp 솔루션의 2세대 설계는 3가지 구성 요소 구성 프로필을 사용하여 검증되었습니다.

구성 프로파일에는 다음이 포함됩니다.

- BeeGFS 관리, 메타데이터 및 스토리지 서비스를 포함한 단일 기본 구성 요소입니다.
- BeeGFS 메타데이터와 스토리지 구성 요소
- BeeGFS 스토리지 전용 구성 요소입니다.

빌딩 블록은 2개의 Mellanox Quantum InfiniBand(MQM8700) 스위치에 연결되었습니다. 10개의 BeeGFS 클라이언트도 InfiniBand 스위치에 연결되었으며 통합 벤치마크 유틸리티를 실행하는 데 사용되었습니다.

다음 그림에서는 NetApp 솔루션의 BeeGFS 검증을 위해 사용되는 BeeGFS 구성을 보여 줍니다.

[]

BeeGFS 파일 스트라이핑

병렬 파일 시스템의 이점은 여러 스토리지 대상 간에 개별 파일을 스트라이핑하는 기능입니다. 이 기능은 동일하거나 다른 기본 스토리지 시스템의 볼륨을 나타낼 수 있습니다.

BeeGFS에서는 디렉토리 및 파일별로 스트라이핑을 구성하여 각 파일에 사용되는 타겟 수를 제어하고 각 파일 스트라이프에 사용되는 청크 크기(또는 블록 크기)를 제어할 수 있습니다. 따라서 서비스를 재구성하거나 다시 시작할 필요 없이 파일 시스템에서 다양한 유형의 워크로드와 I/O 프로필을 지원할 수 있습니다. "begfs -ctl" 명령줄 도구 또는 스트라이핑 API를 사용하는 응용 프로그램을 사용하여 스트라이프 설정을 적용할 수 있습니다. 자세한 내용은 의 BeeGFS 설명서를 참조하십시오 "스트라이핑" 및 "스트라이핑 API".

최상의 성능을 얻기 위해 테스트 중에 스트라이프 패턴을 조정하고 각 테스트에 사용된 매개 변수를 기록하였습니다.

IOR 대역폭 테스트: 여러 클라이언트

IOR 대역폭 테스트는 OpenMPI를 사용하여 합성 I/O 생성기 툴 IOR(에서 제공)의 병렬 작업을 실행했습니다 "HPC GitHub를 참조하십시오") 10개 클라이언트 노드 전체에서 하나 이상의 BeeGFS 구성 요소로 이동합니다. 달리 명시되지 않은 한:

- 모든 테스트는 1MiB 전송 크기의 직접 I/O를 사용했습니다.
- BeeGFS 파일 스트라이핑은 1MB 청크 크기 및 파일당 하나의 타겟으로 설정되었습니다.

다음 매개 변수는 IOR에 사용되었습니다. 세그먼트 수는 구성 요소 1개의 경우 애그리게이트 파일 크기를 5TiB로, 3개의 구성 요소는 40TiB로 유지하도록 조정되었습니다.

```
mpirun --allow-run-as-root --mca btl tcp -np 48 -map-by node -hostfile
10xnodes ior -b 1024k --posix.odirect -e -t 1024k -s 54613 -z -C -F -E -k
```

다음 그림에서는 단일 BeeGFS 기반(관리, 메타데이터 및 스토리지) 구성 요소를 사용한 IOR 테스트 결과를 보여 줍니다.

[]

다음 그림에서는 단일 BeeGFS 메타데이터 + 스토리지 구성 요소를 사용한 IOR 테스트 결과를 보여 줍니다.

[]

다음 그림에서는 단일 BeeGFS 스토리지 전용 구성 요소를 사용한 IOR 테스트 결과를 보여 줍니다.

[]

다음 그림에서는 세 개의 BeeGFS 구성 요소가 포함된 IOR 테스트 결과를 보여 줍니다.

[]

예상한 대로 기본 구성 요소과 후속 메타데이터 + 스토리지 구성 요소 간의 성능 차이는 무시할 수 있습니다. 메타데이터 + 스토리지 구성 요소 및 스토리지 전용 구성 요소를 비교하면 스토리지 대상으로 사용되는 추가 드라이브로 인해 읽기 성능이 약간 향상됩니다. 그러나 쓰기 성능에는 큰 차이가 없습니다. 더 높은 성능을 얻기 위해 여러 구성 요소를 함께 추가하여 성능을 선형 방식으로 확장할 수 있습니다.

IOR 대역폭 테스트: 단일 클라이언트

IOR 대역폭 테스트는 OpenMPI를 사용하여 단일 고성능 GPU 서버를 사용하여 여러 IOR 프로세스를 실행하여 단일 클라이언트에서 얻을 수 있는 성능을 탐색했습니다.

이 테스트는 클라이언트가 Linux 커널 페이지 캐시('tuneFileCacheType=NATIVE')를 사용하도록 구성된 경우 BeeGFS의 다시 읽기 동작 및 성능을 기본 '버퍼링' 설정과 비교합니다.

네이티브 캐싱 모드는 클라이언트의 Linux 커널 페이지 캐시를 사용하므로 네트워크를 통해 다시 전송되는 것이 아니라 로컬 메모리에서 다시 읽기 작업을 수행할 수 있습니다.

다음 다이어그램은 BeeGFS 빌딩 블록 3개와 단일 클라이언트를 사용한 IOR 테스트 결과를 보여 줍니다.

[]

> 이러한 테스트를 위한 BeeGFS 스트라이핑은 파일당 타겟 8개가 포함된 1MB 청크 크기로 설정되었습니다.

기본 버퍼링 모드에서 쓰기 및 초기 읽기 성능이 향상되지만, 동일한 데이터를 여러 번 다시 읽는 워크로드의 경우 네이티브 캐싱 모드에서 성능이 크게 향상됩니다. 이렇게 향상된 다시 읽기 성능은 여러 번의 Epoch에서 동일한 데이터 세트를 여러 번 다시 읽는 딥 러닝과 같은 워크로드에 중요합니다.

메타데이터 성능 테스트

Metadata 성능 테스트는 IOR의 일부로 포함된 MDTest 도구를 사용하여 BeeGFS의 메타데이터 성능을 측정했습니다. 이 테스트에서는 OpenMPI를 사용하여 10개의 클라이언트 노드 모두에서 병렬 작업을 실행했습니다.

다음 매개 변수는 총 프로세스 수가 2배속 단계에서 10개에서 320으로 조정되고 파일 크기가 4K인 벤치마크 테스트를 실행하는 데 사용되었습니다.

```
mpirun -h 10xnodes –map-by node np $processes mdtest -e 4k -w 4k -i 3 -I
16 -z 3 -b 8 -u
```

메타데이터 성능은 먼저 메타데이터 + 스토리지 구성 요소 하나를 측정한 후 추가 구성 요소를 추가하여 성능이 얼마나 향상되는지를 보여 줍니다.

다음 다이어그램은 하나의 BeeGFS 메타데이터 + 스토리지 구성 요소가 포함된 MDTest 결과를 보여 줍니다.

[]

다음 다이어그램은 BeeGFS 메타데이터 + 스토리지 구성 요소 두 개가 포함된 MDTest 결과를 보여 줍니다.

[]

기능 검증

이 아키텍처의 검증 과정에서 NetApp은 다음을 비롯한 여러 기능 테스트를 수행했습니다.

- 스위치 포트를 비활성화하여 단일 클라이언트 InfiniBand 포트에 장애 발생
- 스위치 포트를 비활성화하여 단일 서버 InfiniBand 포트에 장애 발생
- BMC를 사용하여 즉시 서버 전원을 끕니다.
- 노드를 대기 노드에 배치하고 다른 노드에 대한 서비스 장애 조치를 원활히 합니다.
- 노드를 다시 온라인 상태로 전환하고 원래 노드에 서비스를 페일백합니다.
- PDU를 사용하여 InfiniBand 스위치 중 하나의 전원을 끕니다. BeeGFS 클라이언트에 설정된 'sysSessionChecksEnabled:false' 매개 변수를 사용하여 스트레스 테스트가 진행되는 동안 모든 테스트가 수행되었습니다. I/O에 대한 오류나 운영 중단이 관찰되지 않았습니다.

> 알려진 문제가 있습니다(참조) "변경 로그") 기본 인터페이스('connInterfacesFile'에 정의된 대로) 손실 또는 BeeGFS 서버 장애로 인해 BeeGFS 클라이언트/서버 RDMA 연결이 예기치 않게 중단되거나 활성 클라이언트 I/O가 최대 10분 동안 중단되어 다시 시작할 수 있습니다. 이 문제는 계획된 유지 관리를 위해 BeeGFS 노드가 정상적으로 대기 상태가 되거나 TCP가 사용 중인 경우 발생하지 않습니다.

NVIDIA DGX A100 SuperPOD 및 BasePOD 검증

NetApp은 메타데이터와 스토리지 구성 프로필이 적용된 3개의 구성 블록으로 구성된 유사한 BeeGFS 파일 시스템을 사용하여 NVIDIAs DGX A100 SuperPOD에 대한 스토리지 솔루션을 검증했습니다. 검증 노력에는 다양한 스토리지, 머신 러닝 및 딥 러닝 벤치마크를 실행하는 20개의 DGX A100 GPU 서버를 통해 이 NVA에 의해 설명된 솔루션을 테스트하는 작업이 포함되었습니다. NVIDIA DGX A100 SuperPOD에서 사용하도록 인증된 모든 스토리지는 NVIDIA BasePOD 아키텍처에도 자동으로 사용하도록 인증되었습니다.

자세한 내용은 을 참조하십시오 "NetApp을 포함한 NVIDIA DGX SuperPOD" 및 "NVIDIA DGX 베이스POD".

사이징 지침

BeeGFS 솔루션에는 검증 테스트를 기반으로 한 성능 및 용량 사이징에 대한 권장 사항이

포함되어 있습니다.

빌딩 블록 아키텍처의 목표는 특정 BeeGFS 시스템의 요구 사항을 충족하기 위해 여러 빌딩 블록을 추가하여 간편하게 사이징할 수 있는 솔루션을 구축하는 것입니다. 아래 지침에 따라 환경 요구 사항을 충족하는 데 필요한 BeeGFS 빌딩 블록의 양과 유형을 예측할 수 있습니다.

이러한 추정치는 최상의 성능을 제공하는 것으로, 가상 벤치마킹 애플리케이션은 실제 애플리케이션이 사용할 수 없는 방식으로 기본 파일 시스템의 사용을 최적화하기 위해 작성 및 사용됩니다.

성능 사이징

다음 표에는 권장되는 성능 사이징이 나와 있습니다.

구성 프로파일	**1MiB** 읽기	**1MiB**의 쓰기입니다
메타데이터 + 스토리지	62GiBps	21GiBps
스토리지만 해당	64GiBps	21GiBps

메타데이터 용량 사이징 예상치는 "경험 규칙"을 기반으로 하며, 이 경우 500GB의 용량으로 BeeGFS에서 약 1억 5천만 개의 파일을 저장할 수 있습니다. (자세한 내용은 BeeGFS 설명서를 참조하십시오 "시스템 요구 사항"참조)

액세스 제어 목록, 디렉터리별 디렉토리 및 파일 수와 같은 기능을 사용하면 메타데이터 공간이 얼마나 빨리 소비되는지를 알 수 있습니다. 스토리지 용량 추정치는 RAID 6 및 XFS 오버헤드와 함께 사용 가능한 드라이브 용량을 고려합니다.

메타데이터 **+** 스토리지 구성 요소에 대한 용량 사이징

다음 표에는 메타데이터와 스토리지 구성 요소에 권장되는 용량 사이징이 나와 있습니다.

드라이브 크기**(2+2 RAID 1)** 메타데이터 볼륨 그룹	메타데이터 용량**(**파일 수**)**	드라이브 크기**(8 + 2 RAID 6)** 스토리지 볼륨 그룹	스토리지 용량**(**파일 콘텐츠**)**
1.92TB	1,797,120,000	1.92TB	47.91TB
3.84TB	3,594,240,000입니다	3.84TB	95.84TB
7.68TB	7,557,120,000	7.68TB	201.51TB
15.3TB	16,156,800,000	15.3TB	430.84TB

> 메타데이터 및 스토리지 구성 요소의 크기를 조정할 때 더 작은 드라이브를 메타데이터 볼륨 그룹에 사용하는 것과 스토리지 볼륨 그룹을 사용하는 것을 통해 비용을 절감할 수 있습니다.

스토리지 전용 구성 요소에 대한 용량 사이징

다음 표에는 스토리지 전용 구성 요소에 대한 경험 많은 용량 사이징이 나와 있습니다.

드라이브 크기**(10 + 2 RAID 6)** 스토리지 볼륨 그룹	스토리지 용량**(**파일 콘텐츠**)**
1.92TB	59.89TB
3.84TB	119.80TB

드라이브 크기(10 + 2 RAID 6) 스토리지 볼륨 그룹	스토리지 용량(파일 콘텐츠)
7.68TB	251.89TB
15.3TB	538.55TB

> 글로벌 파일 잠금이 활성화되지 않은 경우 기본(첫 번째) 구성 요소에서 관리 서비스를 포함할 때 발생하는 성능 및 용량 오버헤드가 최소화됩니다.

성능 튜닝

BeeGFS 솔루션에는 검증 테스트를 기반으로 한 성능 조정을 위한 권장 사항이 포함되어 있습니다.

BeeGFS가 즉시 사용 가능한 우수한 성능을 제공하기는 하지만, NetApp은 성능을 극대화하기 위해 일련의 권장 튜닝 매개 변수를 개발했습니다. 이 매개 변수는 기본 E-Series 블록 노드의 기능과 공유 디스크 HA 아키텍처에서 BeeGFS를 실행하는 데 필요한 특수 요구사항을 모두 고려합니다.

파일 노드의 성능 튜닝

구성할 수 있는 조정 매개변수는 다음과 같습니다.

1. * 파일 노드의 UEFI/BIOS에서 시스템 설정. * 성능을 극대화하려면 파일 노드로 사용하는 서버 모델에서 시스템 설정을 구성하는 것이 좋습니다. 시스템 설정(UEFI/BIOS) 또는 베이스보드 관리 컨트롤러(BMC)에서 제공하는 Redfish API를 사용하여 파일 노드를 설정할 때 시스템 설정을 구성합니다.

   시스템 설정은 파일 노드로 사용하는 서버 모델에 따라 달라집니다. 사용 중인 서버 모델에 따라 설정을 수동으로 구성해야 합니다. 검증된 Lenovo SR665 파일 노드에 대한 시스템 설정을 구성하는 방법은 을 참조하십시오 "성능을 위해 파일 노드 시스템 설정을 조정합니다".

2. * 필수 구성 매개 변수에 대한 기본 설정 * 필수 구성 매개 변수는 BeeGFS 서비스의 구성 방법과 E-Series 볼륨(블록 장치)의 포맷 및 마운트에 영향을 미칩니다. 이러한 필수 구성 매개 변수는 다음과 같습니다.

   - BeeGFS 서비스 구성 매개 변수입니다

     필요에 따라 구성 매개 변수의 기본 설정을 재정의할 수 있습니다. 특정 워크로드 또는 사용 사례에 맞게 조정할 수 있는 매개 변수는 를 참조하십시오 "BeeGFS 서비스 구성 매개 변수입니다".

   - 볼륨 포맷팅 및 마운팅 매개 변수는 권장 기본값으로 설정되며 고급 사용 사례에만 조정해야 합니다. 기본값은 다음을 수행합니다.

     - 기본 볼륨의 RAID 구성 및 세그먼트 크기와 함께 타겟 유형(예: 관리, 메타데이터 또는 스토리지)을 기준으로 초기 볼륨 포맷을 최적화합니다.
     - 심박동조율기가 각 볼륨을 마운트하는 방법을 조정하여 변경 사항이 즉시 E-시리즈 블록 노드로 플러시되도록 합니다. 이렇게 하면 활성 쓰기가 진행 중일 때 파일 노드가 실패할 때 데이터 손실을 방지할 수 있습니다.

       특정 워크로드 또는 사용 사례에 맞게 조정할 수 있는 매개 변수는 를 참조하십시오 "볼륨 포맷 및 마운팅 구성 매개 변수".

3. * 파일 노드에 설치된 Linux OS의 시스템 설정. * 의 4단계에서 Ansible 인벤토리를 작성할 때 기본 Linux OS 시스템 설정을 재정의할 수 있습니다 "Ansible 인벤토리를 작성합니다".

기본 설정을 사용하여 NetApp 기반 BeeGFS 솔루션을 검증했지만 특정 워크로드 또는 사용 사례에 맞게 수정할 수 있습니다. 변경할 수 있는 Linux OS 시스템 설정의 예는 다음과 같습니다.

- E-Series 블록 장치의 I/O 큐

  BeeGFS 타겟으로 사용되는 E-Series 블록 디바이스에서 I/O 큐를 구성하여 다음을 수행할 수 있습니다.

  - 장치 유형(NVMe, HDD 등)에 따라 스케줄링 알고리즘을 조정합니다.
  - 미결 요청 수를 늘립니다.
  - 요청 크기를 조정합니다.
  - 미리 읽기 동작 최적화
- 가상 메모리 설정.

  최적의 스트리밍 성능을 위해 가상 메모리 설정을 조정할 수 있습니다.

- CPU 설정

  최대 성능을 위해 CPU 주파수 조절기 및 기타 CPU 구성을 조정할 수 있습니다.

- 읽기 요청 크기입니다.

  Mellanox HCA의 최대 읽기 요청 크기를 늘릴 수 있습니다.

블록 노드의 성능 튜닝

특정 BeeGFS 빌딩 블록에 적용되는 구성 프로필을 기준으로 블록 노드에 구성된 볼륨 그룹이 약간 변경됩니다. 예를 들어, 24-드라이브 EF600 블록 노드는 다음과 같습니다.

- BeeGFS 관리, 메타데이터 및 스토리지 서비스를 비롯한 단일 기본 구성 요소:
  - BeeGFS 관리 및 메타데이터 서비스를 위한 1 x 2 + 2 RAID 10 볼륨 그룹
  - BeeGFS 스토리지 서비스용 2x 8+2 RAID 6 볼륨 그룹
- BeeGFS 메타데이터 + 스토리지 구성 요소:
  - BeeGFS 메타데이터 서비스용 2 + 2 RAID 10 볼륨 그룹 1개
  - BeeGFS 스토리지 서비스용 2x 8+2 RAID 6 볼륨 그룹
- BeeGFS 스토리지 전용 구성 요소:
  - BeeGFS 스토리지 서비스용 10+2 RAID 6 볼륨 그룹 2개

> BeeGFS는 스토리지 대비 관리 및 메타데이터를 위해 스토리지 공간이 훨씬 적게 요구되므로 RAID 10 볼륨 그룹에 더 작은 드라이브를 사용하는 옵션이 있습니다. 작은 드라이브는 가장 바깥쪽 드라이브 슬롯에 장착해야 합니다. 자세한 내용은 를 참조하십시오 "배포 지침".

이러한 모든 설정은 Ansible 기반 배포를 통해 구성되며, 다음은 성능/동작을 최적화하기 위해 일반적으로 권장되는 몇 가지 다른 설정과 함께 제공됩니다.

- 글로벌 캐시 블록 크기를 32KiB로 조정하고 요구 기반 캐시 플러시를 80%로 조정합니다.

- 자동 로드 밸런싱 비활성화(컨트롤러 볼륨 할당이 의도한 대로 유지되는지 확인)
- 읽기 캐싱 설정 및 미리 읽기 캐싱 해제
- 미러링으로 쓰기 캐시를 설정하고 배터리 백업이 필요하므로 블록 노드 컨트롤러의 장애가 발생해도 캐시가 유지됩니다.
- 드라이브가 볼륨 그룹에 할당되는 순서를 지정하여 사용 가능한 드라이브 채널 간에 I/O의 균형을 조정합니다.

저작권 정보

Copyright © 2022 NetApp, Inc. All rights reserved. 미국에서 인쇄됨 본 문서의 어떤 부분도 저작권 소유자의 사전 서면 승인 없이는 어떠한 형식이나 그래픽, 전자적 또는 기계적 수단(사진 복사, 레코딩 등)으로도 저작권 소유자의 사전 서면 승인 없이 전자 검색 시스템에 저장 또는 저장.

NetApp이 저작권을 가진 자료에 있는 소프트웨어에는 아래의 라이센스와 고지 사항이 적용됩니다.

본 소프트웨어는 NetApp에 의해 "있는 그대로" 제공되며 상품성 및 특정 목적에 대한 적합성에 대한 명시적 또는 묵시적 보증을 포함하여 이에 제한되지 않고, 어떠한 보증도 하지 않습니다. NetApp은 대체품 또는 대체 서비스의 조달, 사용 불능, 데이터 손실, 이익 손실, 또는 파생적 손해(소계 물품 또는 서비스의 조달, 사용 손실, 데이터 또는 수익 손실, 계약, 엄격한 책임 또는 불법 행위(과실 또는 그렇지 않은 경우)에 관계없이 어떠한 책임도 지지 않으며, 이는 이러한 손해의 가능성을 사전에 알고 있던 경우에도 마찬가지입니다.

NetApp은 본 문서에 설명된 제품을 언제든지 예고 없이 변경할 권리를 보유합니다. NetApp은 NetApp의 명시적인 서면 동의를 받은 경우를 제외하고 본 문서에 설명된 제품을 사용하여 발생하는 어떠한 문제에도 책임을 지지 않습니다. 본 제품의 사용 또는 구입의 경우 NetApp에서는 어떠한 특허권, 상표권 또는 기타 지적 재산권이 적용되는 라이센스도 제공하지 않습니다.

본 설명서에 설명된 제품은 하나 이상의 미국 특허 또는 해외 특허, 해외 특허, 해외 특허, 해외 특허, 해외 특허, 해외 특허, 해외 특허, 미국 출원 중인 특허로 보호됩니다.

권리 제한 표시: 정부에 의한 사용, 복제 또는 공개에는 DFARS 252.277-7103(1988년 10월) 및 FAR 52-227-19(1987년 6월)의 기술 데이터 및 컴퓨터 소프트웨어의 권리(Rights in Technical Data and Computer Software) 조항의 하위 조항 (c)(1)(ii)에 설명된 제한사항이 적용됩니다.

상표 정보

NETAPP, NETAPP 로고 및 에 나열된 마크는 NetApp에 있습니다 http://www.netapp.com/TM 는 NetApp, Inc.의 상표입니다. 기타 회사 및 제품 이름은 해당 소유자의 상표일 수 있습니다.